School Management System - OOP Model

1. Introduction

This document describes an Object-Oriented Programming (OOP) model for a School Management System.

The key components include Students, Teachers, Courses, and Departments, along with their interactions.

Core OOP principles such as Abstraction, Encapsulation, Inheritance, and Polymorphism are utilized to build

a modular and maintainable system.

2. Class Definitions

2.1 Person Class (Base Class)

```python
class Person:
    def __init__(self, name, age, gender, id_number):
        self.name = name
        self.age = age
        self.gender = gender
        self.id_number = id_number

    def display_info(self):
        print(f"Name: {self.name}, Age: {self.age}, Gender: {self.gender}, ID: {self.id_number}")
```

```

2.2 Student Class (Inheriting from Person)

```python
class Student(Person):
    def __init__(self, name, age, gender, id_number, student_id, courses=None):
        super().__init__(name, age, gender, id_number)
        self.student_id = student_id
        self.courses = courses if courses else []

    def enroll_course(self, course):
        self.courses.append(course)

    def display_courses(self):
        print(f"{self.name}'s Courses: {', '.join([course.course_name for course in self.courses])}")
```

2.3 Teacher Class (Inheriting from Person)

```python
class Teacher(Person):
    def __init__(self, name, age, gender, id_number, teacher_id, department):
        super().__init__(name, age, gender, id_number)
        self.teacher_id = teacher_id
        self.department = department

    def assign_course(self, course):
```
```

```
        course.teacher = self
```

2.4 Course Class

```python
class Course:
    def __init__(self, course_name, course_code, teacher=None):
        self.course_name = course_name
        self.course_code = course_code
        self.teacher = teacher
        self.students = []

    def add_student(self, student):
        self.students.append(student)
        student.enroll_course(self)
```

3. Sample Usage

```python
# Create Department
math_department = Department("Mathematics")

# Create Teacher and Assign to Department
teacher1 = Teacher("Alice", 40, "Female", "T001", "TEA123", math_department)
math_department.add_teacher(teacher1)
```

```
# Create Course and Assign Teacher

course1 = Course("Calculus", "MATH101")

teacher1.assign_course(course1)

math_department.add_course(course1)



# Create Students and Enroll in Course

student1 = Student("Bob", 18, "Male", "S001", "STU456")

student2 = Student("Clara", 19, "Female", "S002", "STU789")



course1.add_student(student1)

course1.add_student(student2)



# Display Information

course1.display_course_info()

math_department.display_department_info()

student1.display_courses()
```

4. OOP Concepts in Action

1. **Abstraction**: Classes hide complex data and expose only relevant methods and attributes.

2. **Encapsulation**: Data members are bundled into specific classes and hidden from outside access.

3. **Inheritance**: `Student` and `Teacher` inherit from the `Person` class, reducing redundancy.

4. **Polymorphism**: Methods like `display_info()` behave differently based on the class instance.